

Software Requirements Specification

Initiative Name: Secure API Boundary Layer (SABL)

Author: Benjamin Soto-Roberts

Created: July 21, 2026



Introduction

1.1 Purpose/Value Proposition

This project serves as a training and portfolio exercise designed to simulate a small-scale, production-grade multi-tenant Identity and Access Management (IAM) component, with an emphasis on secure application development practices.

1.2 Audience

This document is intended for developers, security engineers, and internal stakeholders responsible for maintaining or integrating with the IAM service.

1.3 Scope

In Scope

- Multi-tenant organization model with enforced tenant isolation
- Three-tier role-based access control (RBAC): Platform Admin, Org Superuser, User
- Org Superuser-managed user provisioning, scoped to their own organization
- HTTP Basic authentication
- HTTPS/TLS enforced; self-signed certificate for local development

- Secure endpoint protection (authentication and authorization)
- Standardized HTTP status codes and response headers (RFC 9110)
- Standardized error responses using Problem Details (RFC 9457)
- Brute-force and rate-limit protections
- Structured, OWASP-aligned security event logging
- Secure SDLC tooling: dependency/SCA scanning, SAST, and code coverage gates

Out Of Scope

- Self-service user registration
- JWT / token-based authentication
- Refresh tokens
- Stateful sessions
- Cross-tenant resource sharing
- Secrets scanning and container/image scanning (future work)

Overall Description

2.1 Overview

The system manages one or more tenant organizations, each with an isolated set of users and roles. Access is governed by a three-tier RBAC model:

- Platform Admin – manages organizations at the platform level; not scoped to a single tenant.
- Org Superuser – manages users and role assignments within their own organization only.
- User – provisioned by an Org Superuser; restricted to read access on their own profile within their own organization.

All requests are authenticated using HTTP Basic credentials transported exclusively over TLS. Authorization is enforced per-request against both the caller's role and their organization (tenant) membership, so that no principal can read or modify data belonging to another organization regardless of role.

Requirements

3.1 Functional Requirements

FR-1. When a client sends a valid HTTP Basic Authorization header with valid credentials to an authenticated endpoint, the server authenticates the client and establishes their identity, role, and organization context for the request.

FR-2. When an authenticated Platform Admin requests platform-level organization data, the server returns a 200 OK status with the requested organization data, unscoped by tenant.

FR-3. When an authenticated Org Superuser manages (creates, updates, or deactivates) a user, the server permits the operation only if the target user belongs to the Superuser's own organization; otherwise the server returns a 403 Forbidden status with a standardized problem detail response.

FR-4. When an authenticated User requests their own profile, the server returns a 200 OK status with the user's non-sensitive profile details and granted role, scoped to their own organization.

FR-5. When a client sends an Authorization header with invalid or missing credentials to a protected endpoint, the server rejects the client and returns a 401 Unauthorized status with a standardized problem detail response.

FR-6. When an authenticated principal requests a resource for which their role does not grant access, the server returns a 403 Forbidden status with a standardized problem detail response, after authentication succeeds and authorization fails.

FR-7. When an authenticated principal requests a resource belonging to an organization other than their own, the server denies the request regardless of role, returning a 403 Forbidden status with a standardized problem detail response.

3.2 Non-Functional Requirements

NFR-1. The API must enforce HTTPS. Plaintext HTTP requests must be rejected with HTTP 421 (Misdirected Request). The server must only accept connections on port 443 in production; local development uses a self-signed certificate. HSTS (Strict-Transport-Security) must be applied to all responses.

NFR-2. All authentication and authorization events must produce a structured log entry, aligned to the OWASP Logging Cheat Sheet, containing: outcome (success/failure), principal identifier, organization (tenant) context, role, timestamp (ISO 8601), and source IP. Log entries must not include plaintext credentials. Source IP fields must be treated as PII in accordance with applicable data handling policy.

NFR-3. Passwords must be stored as one-way hashes using Bcrypt with a minimum work factor of 12. Passwords are never stored in plaintext or using reversible encryption.

NFR-4. HTTP responses must conform to RFC 9110 semantics for the applicable method and status code – including required and recommended response headers (e.g., a 201 Created response must include a Location header referencing the created resource's URI).

NFR-5. All error responses must conform to IETF RFC 9457 (Problem Details for HTTP APIs).

NFR-6. All database connections must use a dedicated, least-privilege role. This role must be scoped to only the permissions required for IAM operations. Root or superuser database credentials must never be used by the application.

NFR-7. The API must enforce a rate limit on authentication attempts per source IP. After a configurable threshold of consecutive failed attempts, the source IP must be temporarily blocked and a 429 Too Many Requests response returned.

NFR-8. All data access must be scoped by organization (tenant) identifier at the data access layer. No query may return or modify records across organization boundaries, regardless of the caller's role.

NFR-9. The build pipeline must enforce, at minimum: dependency/SCA scanning with the build failing on discovered CVEs of severity 4.0 (CVSS) or higher; static analysis (SAST) via SonarQube with an enforced quality gate; and code coverage reporting via JaCoCo. Supplemental SAST via Semgrep is run outside the build pipeline and is not currently build-gated.

Standards

RFC 6797 – HTTP Strict Transport Security (HSTS), supports NFR-1: <https://www.rfc-editor.org/rfc/rfc6797.html>

OWASP Logging Cheat Sheet, supports NFR-2:
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

OWASP Logging Vocabulary Cheat Sheet, supports NFR-2 structured event fields:
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Vocabulary_Cheat_Sheet.html

OWASP Password Storage Cheat Sheet, supports NFR-3 bcrypt rationale:
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

RFC 9110 – HTTP Semantics, supports NFR-5: <https://www.rfc-editor.org/rfc/rfc9110.html>

RFC 9457 – Problem Details for HTTP APIs, supports NFR-5: <https://www.rfc-editor.org/rfc/rfc9457.html>

OWASP Authentication Cheat Sheet, supports NFR-6 rate-limiting design:
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html